Deliverable — Project Zero's "MTE As Implemented" (T3)

Mark Brand's three-part Project Zero series (August 2023) treats MTE as a security mitigation, not a debugging tool, and lays out a clean taxonomy of how it gets bypassed. I read Parts 2 and 3 in full. Part 1 (hardware limits) remains.

In plain English first

Google's own elite bug-hunting team (Project Zero) wrote up how well MTE actually holds up in the real world. **Their verdict:** MTE is not an unbreakable wall — it's a speed bump, and how tall the bump is depends on *where* you hit it. It's tallest against a remote attacker sending you a malicious file (like our dav1d/Skia targets), and shortest against an attacker who can already run code on your phone. This tab is the single best justification for picking a remote file-parser as our target.

Word bank (terms in this tab)

Term	Plain meaning
Mitigation	A defense that makes a bug *harder to exploit* without fixing the bug itself. MTE is one.
Known-tag bypass	Attacker *learns the color first* (usually via a timing trick), then their bad access matches and never crashes.
Unknown-tag bypass	Attacker wins *even with the wrong color*, by exploiting a gap (e.g. the async delay).
Side channel	Learning a secret indirectly — e.g. from how long something takes — instead of reading it directly.
Speculative execution	The CPU guessing ahead and running code before it's sure it's needed. Attackers abuse the leftovers to leak secrets.
SYNC / ASYNC	MTE crashing *instantly* on a bad access vs. *a moment later*. The delay in ASYNC is a usable gap.

Term	Plain meaning
Parser	Code that reads a complex input format (video, image, message). The classic remote attack surface.
Zygote	The Android process all apps are forked from. They share a memory layout, which creates a weakness.

The two bypass classes (adopt this vocabulary)

- **Known-tag-bypass.** The attacker learns the tag, so their invalid access carries the *correct* tag and never faults. Tag confidentiality is the whole game; a speculative side channel breaks it. TikTag and PACMAN are instances.
- **Unknown-tag-bypass.** The attacker succeeds *despite* a wrong tag, by exploiting an implementation limit (e.g. completing the exploit during the async window before the fault is delivered).

Headline conclusion (paraphrased): using standard memory tagging as a "hard probabilistic mitigation" is not generally possible, because wherever an attacker can build a speculative side channel, known-tag-bypasses are a fundamental weakness. A citable statement of the project's core premise, from Google's own red team.

SYNC vs ASYNC, sharpened

- **ASYNC:** results of invalid reads and effects of invalid writes are *architecturally observable* before the fault arrives — what makes unknown-tag-bypasses practical. Demonstrated against async-MTE.
- **SYNC:** invalid reads/writes should *not* be observable, so unknown-tag-bypasses should be rare. SYNC is necessary for any "hard" mitigation.
- Even a "soft" (async) mitigation still inconveniences attackers and raises detection risk. "Bypassable" is not "worthless."

The per-context difficulty table (gold for target selection)

Context	Known-tag (async / sync)	Unknown-tag (sync)	Takeaway
Chrome renderer (JS → compromised renderer)	Trivial / trivial	Rare	JS avoids syscalls; soft time constraint easy. MTE barely bites.
Chrome IPC sandbox escape	Possible / possible	Rare	Harder; some IPC batches without syscalls.
Android Binder sandbox escape	Depends on service	Rare	Mandatory `ioctl(BINDER_WRITE_READ)` between messages; exploit must finish inside one.
Android messaging app (remote one-shot)	Likely impossible	Good-enough bugs very rare	**MTE bites hardest.** Remote attacker can't easily build a side channel; needs a very good one-shot bug in complex file-format parsing.

MTE is strongest against a remote attacker hitting a parser, weakest against a local attacker who can run code and build side channels. So a remote-reachable file-format parser is the most interesting place to test an agent — and the classic zero-click surface.

Two concrete Android weaknesses Brand names

1. **`.data` section pointers are not tagged.** With Android's zygote architecture (all apps fork from one process, shared layout), pointer-forging between some contexts is easy — useful for second-order corruption like type confusion. A coverage gap that needs no leak.
2. **Repeated-attempt channels.** For a messaging app, known-tag-bypasses are unlikely *unless the app allows repeated attempts until the tag is correct.* The restart question, as a concrete app property to check on any target.

Kernel-specific problems (Part 3)

- **Tag 0b1111 bypasses checks.** The kernel needs `TCR_ELx.TCMA1` set, so any pointer tagged 0b1111 is dereferenced with no tag check. Forge a tag-15 pointer and you sidestep MTE. (In userspace Scudo it's tag 0 that's special; in the kernel it's tag 15.)
- **Speculative side channels break kernel tag confidentiality too**, plus kernel pointer-equality leaks (`kcmp`, epoll) and `TYPESAFE_BY_RCU` regions that require UAF by design and can't be tagged.

What this changes

- Adopt the known-tag / unknown-tag vocabulary in the writeup.
- Target selection should lean toward a remote-reachable parser — where MTE is a real obstacle and an agent's bypass is a meaningful result.
- SYNC mode is non-negotiable for the "does MTE stop this" question.
- The `.data`/zygote pointer-forging gap and the "repeated attempts" property are concrete things to check on any candidate.